
FACULTAD DE INGENIERÍA EN ELECTRICIDAD Y COMPUTACIÓN

Ingeniería en Ciencias de la Computación
Software Engineering II — 2026

Reporte de Práctica

INTEGRACIÓN CONTINUA

Configuración de un Pipeline de CI/CD con GitHub Actions
Aplicado a CaféLibro — Library Loan Manager

INTEGRANTES DEL GRUPO

#	Nombre completo	Feature
1	Luis Romero	Feature 1
2	Mariu Andrade	Feature 2
3	Javier Gutiérrez	Feature 3
4	Michael Poveda	Feature 4
5	Juan Severino	Feature 5
6	Nahin Cevallos	Feature 6

Índice general

1. Introducción	2
2. Desarrollo	3
2.1. Repositorio compartido	3
2.2. Workflow de Integración Continua	3
2.3. Protección de la rama principal	5
2.4. Desarrollo de features en paralelo	5
2.5. Verificación del pipeline ante cambios fallidos	6
2.6. Automatización del release	6
2.7. Decisión sobre Continuous Deployment	6
2.8. Preguntas	7
2.8.1. Valor de la integración continua en un proyecto de equipo . . .	7
2.8.2. Comparación de herramientas CI/CD	8
2.8.3. Herramienta elegida para el proyecto	8
2.8.4. CI y metodologías ágiles	8
3. Conclusiones y Recomendaciones	10

1. Introducción

La integración continua es una práctica de desarrollo en la que cada miembro del equipo fusiona sus cambios a un repositorio compartido con frecuencia, y cada fusión dispara automáticamente la construcción del proyecto y la ejecución de las pruebas. El objetivo es detectar errores lo antes posible: cuando los cambios son pequeños y recientes, encontrar el origen de un fallo es sencillo y corregirlo tiene un costo bajo. Sin esta práctica, los equipos tienden a integrar tarde, los conflictos se acumulan y los defectos se vuelven difíciles de rastrear.

La integración continua es también la base de la entrega continua y el despliegue continuo. La entrega continua automatiza el proceso de liberación para que cualquier versión que pase el pipeline pueda ser publicada en cualquier momento con aprobación humana. El despliegue continuo elimina esa aprobación y envía cada cambio directamente a los usuarios.

El objetivo de esta práctica es construir un pipeline de integración continua sobre un repositorio compartido en GitHub usando GitHub Actions, aplicado al desarrollo colaborativo de CaféLibro, una herramienta de gestión de préstamos de biblioteca donde cada integrante desarrolló una funcionalidad en rama separada e integró sus cambios mediante pull requests revisados.

2. Desarrollo

2.1. Repositorio compartido

El repositorio del equipo se encuentra disponible en: <https://github.com/lromeror/IntegracionContinuaTaller>

La siguiente tabla resume las funcionalidades implementadas y el integrante responsable de cada una:

#	Integrante	Feature
1	Luis Romero	Registrar un miembro con nombre e identificador único
2	Mariu Andrade	Prestar un libro a un miembro
3	Javier Gutiérrez	Devolver un libro, marcándolo disponible nuevamente
4	Michael Poveda	Listar los libros que un miembro tiene en préstamo
5	Juan Severino	Reportar los préstamos vencidos dado una fecha límite
6	Nahin Cevallos	Incluir un libro en el catálogo, con su título y un código único.

2.2. Workflow de Integración Continua

El archivo `.github/workflows/ci.yml` define el pipeline de integración continua. Se ejecuta automáticamente en cada push y en cada pull request dirigido a la rama `main`, instalando las dependencias y corriendo la suite de pruebas completa en un entorno limpio de Ubuntu.

Listing 2.1: Archivo `ci.yml` del pipeline de CI

```

1 name: ci
2
3 on:
4   push:
5     branches: [main]
6   pull_request:
7     branches: [main]
8
9 jobs:
10  test:
11    runs-on: ubuntu-latest
12    steps:
13      - uses: actions/checkout@v4
14      - uses: actions/setup-python@v5
15        with:
16          python-version: "3.12"
17      - run: pip install -e .
18      - run: pip install pytest
19      - run: pytest -q

```

La Figura 2.1 muestra una ejecución exitosa del workflow en la pestaña Actions, correspondiente al pull request Feature/register member #2.

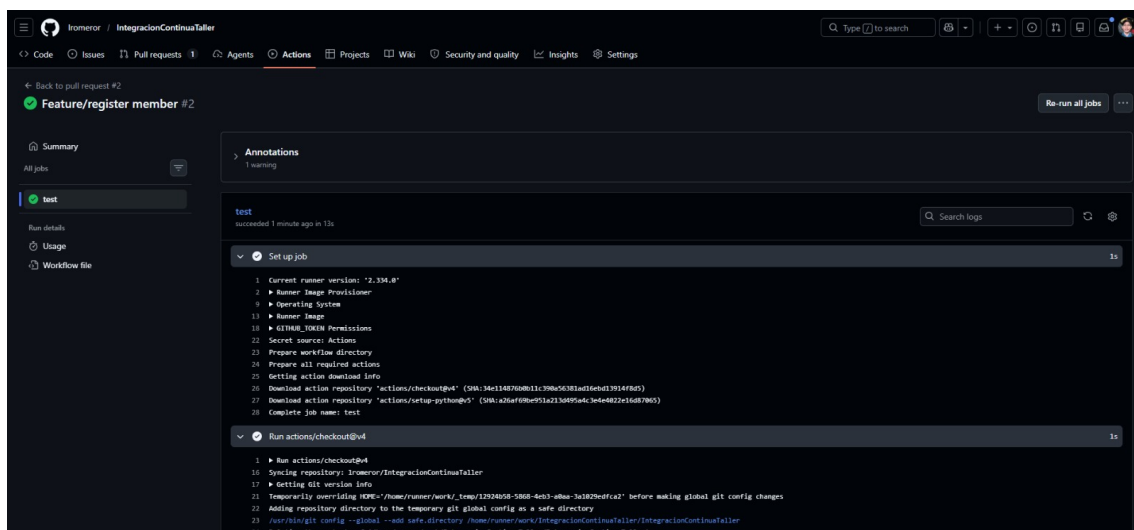


Figura 2.1: Ejecución exitosa del workflow de CI en GitHub Actions

2.3. Protección de la rama principal

Se configuró un ruleset llamado **protect-main** en los ajustes del repositorio con estado *Active*. La rama **main** fue definida como rama objetivo, de modo que ningún cambio puede fusionarse sin que el workflow de CI haya pasado y sin contar con al menos una revisión aprobatoria. La Figura 2.2 muestra la configuración aplicada.

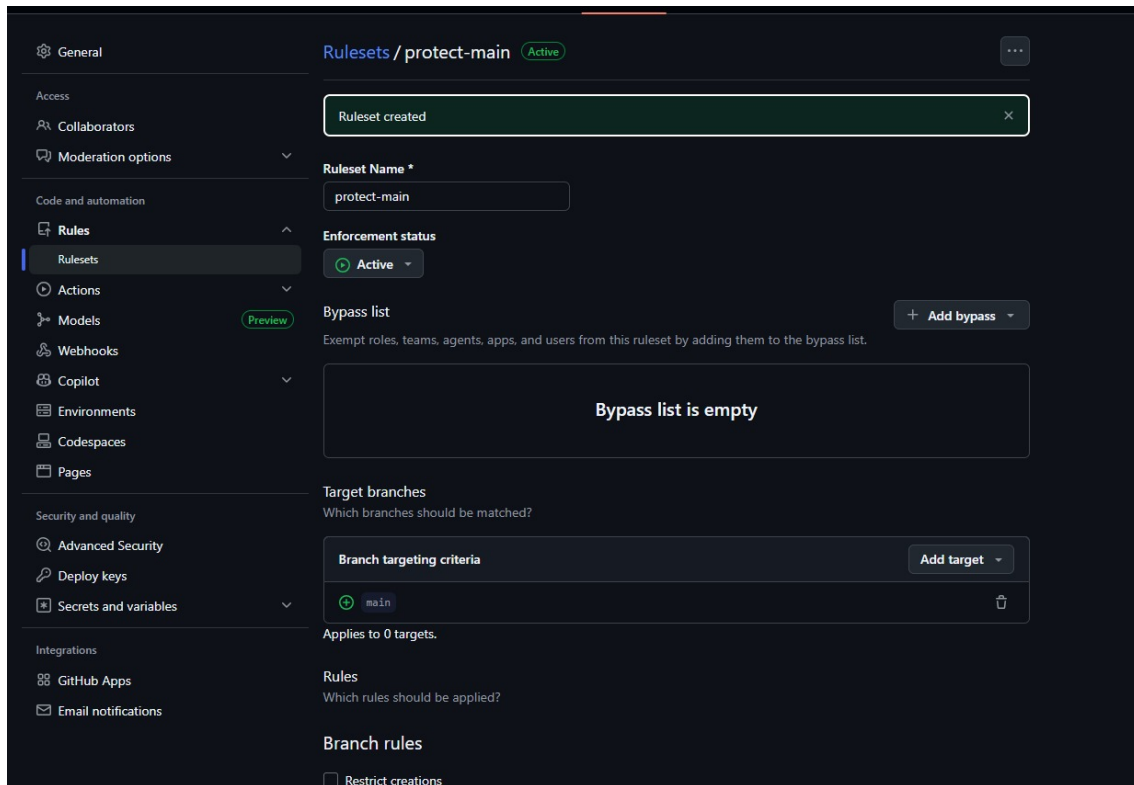


Figura 2.2: Configuración del ruleset protect-main en GitHub

2.4. Desarrollo de features en paralelo

Cada integrante trabajó en su propia rama con el prefijo **feature/**, abrió un pull request hacia **main** y recibió revisión de otro miembro del equipo antes de hacer merge. La Figura 2.3 corresponde al pull request de la Feature 2 (Mariu Andrade), donde se puede observar que el merge estaba bloqueado por requerir al menos una revisión aprobatoria, y que el check de CI aún estaba en espera de reporte de estado.

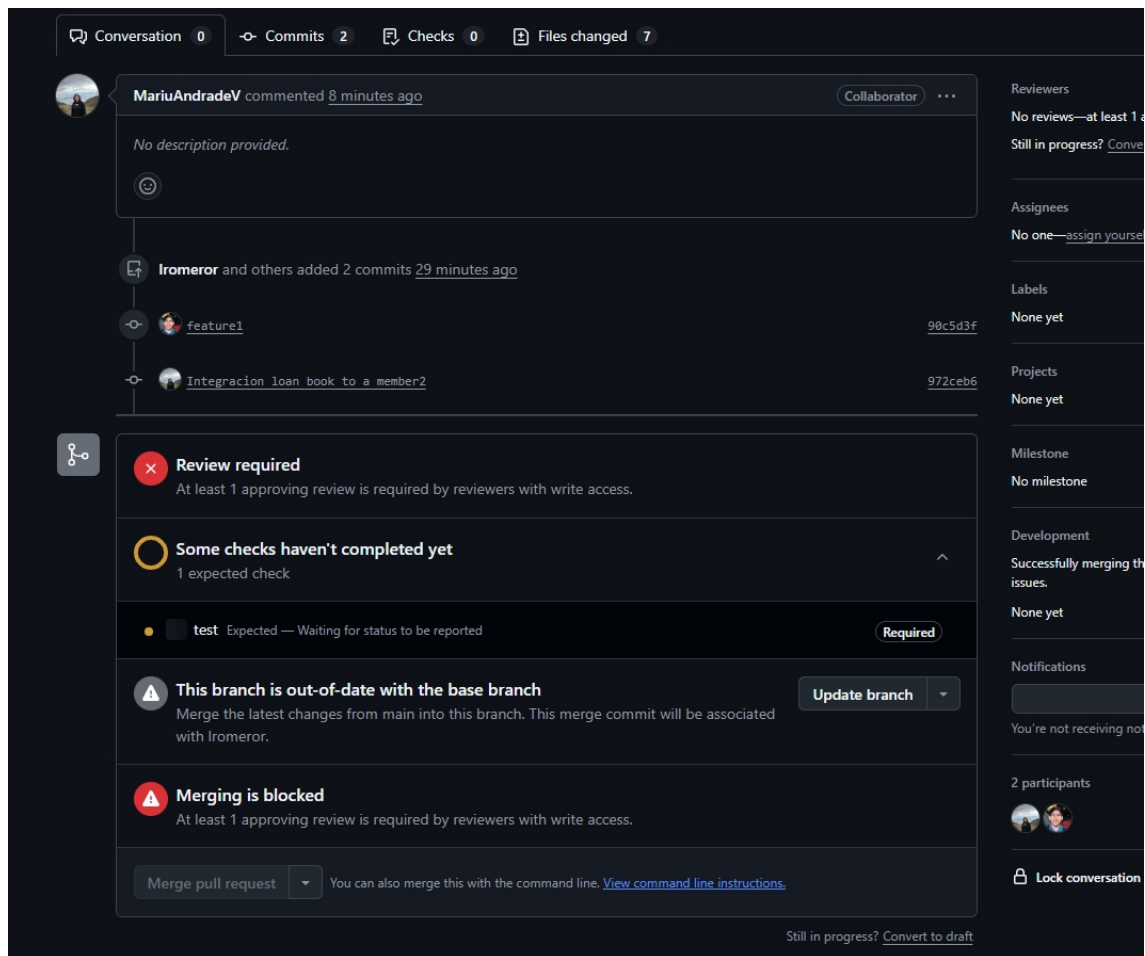


Figura 2.3: Pull request bloqueado por revisión requerida y check pendiente

2.5. Verificación del pipeline ante cambios fallidos

Para verificar que el pipeline bloquea correctamente los cambios que rompen pruebas existentes, se realizó un commit deliberado que alteraba el resultado esperado de una prueba. El pull request correspondiente mostró el check en rojo e impidió el merge. Tras corregir el cambio en un commit posterior, el check volvió a verde y el merge quedó habilitado.

2.6. Automatización del release

2.7. Decisión sobre Continuous Deployment

¿Qué habría que cambiar en el workflow para activar CD?

Actualmente el pipeline construye y prueba cada cambio, pero no publica nada de forma automática. Para activar despliegue continuo habría que agregar un job

adicional en `ci.yml` que se ejecute únicamente cuando el merge llega a `main` y el job de pruebas pasa. Ese job publicaría el paquete en el entorno de producción sin intervención humana, eliminando el paso de aprobación manual que hoy existe en la etapa de release.

¿Qué podría llegar a los usuarios si una prueba no detecta un error?

CaféLibro maneja reglas de negocio concretas: un libro no puede estar prestado a dos miembros a la vez, y un miembro no puede tener más de tres libros simultáneamente. Si una prueba no cubre un caso borde de estas reglas, un bug podría llegar directamente a los usuarios. Por ejemplo, un error en la feature de préstamos podría permitir registrar un libro como disponible cuando en realidad ya está asignado, corrompiendo el estado del archivo JSON que persiste los datos entre sesiones.

Salvaguardas que usan los equipos reales

Antes de confiar en despliegue continuo, los equipos suelen aplicar al menos tres mecanismos. El primero es el despliegue en canario (*canary deployment*): en lugar de enviar cada cambio al cien por ciento de los usuarios de golpe, se libera primero a un porcentaje pequeño y se monitorea el comportamiento antes de expandir el rollout [3]. El segundo es contar con un entorno de staging que replica producción: el pipeline despliega ahí primero y solo avanza si las métricas son estables. El tercero es tener un mecanismo de rollback automatizado que revierta el despliegue si las alarmas de producción superan un umbral definido.

Decisión del equipo

El equipo decidió no activar despliegue continuo en esta etapa. La cobertura de pruebas actual es suficiente para validar el flujo principal, pero no cubre todos los casos borde de las reglas de la librería. Activar CD sin ese nivel de cobertura significa asumir un riesgo que no está justificado para una herramienta que maneja estado persistente en un archivo compartido. Cambiaríamos de opinión si la cobertura de pruebas superara el 90 %, si contáramos con un entorno de staging configurado y si hubiera un proceso de rollback probado.

2.8. Preguntas

2.8.1. Valor de la integración continua en un proyecto de equipo

La integración continua vale el esfuerzo de configuración porque el problema que resuelve crece con el tiempo. En un repositorio compartido sin CI, cada desarrollador

trabaja en su rama durante días o semanas, y cuando llega el momento de integrar, los cambios de todos chocan al mismo tiempo. Resolver esos conflictos tarda más que haberlos evitado, y los bugs que aparecen son difíciles de ubicar porque no hay un único cambio responsable sino una acumulación de varios. Con CI, cada push pequeño se verifica solo, el feedback llega en minutos y el costo de corregir es mínimo. Lo que falla sin CI no es solo el código: falla la visibilidad del equipo sobre el estado real del proyecto.

2.8.2. Comparación de herramientas CI/CD

Jenkins es una herramienta de código abierto que el equipo instala y administra en su propia infraestructura. La configuración del pipeline se define en un archivo llamado **Jenkinsfile** que vive en el repositorio, usando una sintaxis declarativa o scripted en Groovy. El build corre en el servidor propio o en agentes que el equipo configure, lo que da control total pero requiere mantener la infraestructura [4].

GitLab CI está integrado directamente en la plataforma GitLab. El pipeline se configura en un archivo `.gitlab-ci.yml` en la raíz del repositorio, con una sintaxis YAML similar a GitHub Actions. Los builds corren en runners que pueden ser compartidos (provistos por GitLab) o propios. La ventaja principal es que el código, el pipeline, los issues y los merge requests viven en el mismo lugar sin necesidad de integraciones externas [5].

CircleCI es un servicio en la nube que se conecta al repositorio mediante OAuth. La configuración va en `.circleci/config.yml` y los builds corren en contenedores administrados por CircleCI, aunque también soporta runners propios. Tiene una capa gratuita generosa y una interfaz clara para visualizar pipelines [6].

2.8.3. Herramienta elegida para el proyecto

Para este proyecto elegiríamos GitHub Actions. El código ya vive en GitHub, por lo que no hay que configurar integraciones externas ni manejar credenciales adicionales: el workflow tiene acceso al repositorio de forma nativa. La sintaxis YAML es directa, la documentación es extensa y los runners de Ubuntu gratuitos son suficientes para correr pytest sin costo. Jenkins requeriría mantener un servidor propio, y aunque GitLab CI es igual de cómodo, implicaría mover el repositorio o duplicarlo. Para un proyecto de curso donde el tiempo de setup importa, GitHub Actions es la opción más razonable.

2.8.4. CI y metodologías ágiles

Las metodologías ágiles organizan el trabajo en iteraciones cortas, generalmente de una a dos semanas, al final de las cuales el equipo debe tener software funcional y

potencialmente entregable. CI encaja directamente con ese modelo: si cada cambio se integra y verifica de forma continua, al terminar la iteración el equipo no necesita una fase de integración separada porque el código ya fue consolidado y probado durante el sprint. Sin CI, el último día de cada iteración suele convertirse en una sesión de resolución de conflictos en lugar de una demo. Con CI, la integración ocurre en paralelo al desarrollo y el cierre de cada sprint es predecible.

3. Conclusiones y Recomendaciones

La práctica permitió experimentar de forma directa los principios de integración continua sobre un repositorio real con múltiples colaboradores. Configurar el pipeline desde cero dejó en claro que el valor de CI no está en la herramienta sino en la disciplina que impone: integrar frecuentemente, no avanzar sobre un build roto y revisar el código de otros antes de que llegue a la rama principal.

Como recomendaciones, el equipo identificó tres aspectos a mejorar en futuras iteraciones. Primero, ampliar la cobertura de pruebas para incluir casos borde de las reglas de negocio, especialmente los límites de préstamos simultáneos. Segundo, agregar un paso de análisis estático de código al pipeline para detectar problemas de estilo y posibles errores antes de que los revisores los vean. Tercero, configurar notificaciones automáticas al canal del equipo cuando un build en `main` falla, para que la corrección no dependa de que alguien recuerde revisar la pestaña de Actions.

Bibliografía

- [1] Fowler, M. (2006). *Continuous Integration*. martinowler.com. Recuperado de <https://martinfowler.com/articles/continuousIntegration.html>
- [2] GitHub. (2024). *Understanding GitHub Actions*. GitHub Docs. Recuperado de <https://docs.github.com/en/actions/learn-github-actions/understanding-github-actions>
- [3] Sato, D. (2014). *CanaryRelease*. martinowler.com. Recuperado de <https://martinfowler.com/bliki/CanaryRelease.html>
- [4] Jenkins. (2024). *Jenkins User Documentation*. Recuperado de <https://www.jenkins.io/doc/>
- [5] GitLab. (2024). *GitLab CI/CD*. Recuperado de <https://docs.gitlab.com/ee/ci/>
- [6] CircleCI. (2024). *CircleCI Documentation*. Recuperado de <https://circleci.com/docs/>